

Security Fundamentals · Section 7

Access Control Basics

By TOUCH DEN · November 11, 2026

Contents

What You Will Learn 1

Before You Start 1

1. Authentication vs. Authorization 1

2. Least Privilege 1

3. Separation of Duties 2

4. Access Control Models 3

5. Users, Groups, and Roles 3

6. Privilege Creep 4

7. Access Control in Practice 4

Common Mistakes 5

Summary 5

Check Your Understanding 5

Practice 6

What You Will Learn

- The difference between authentication and authorization.
- The principle of least privilege, the most important idea in access control.
- Separation of duties, and why it stops fraud.
- The main access control models: DAC, MAC, RBAC, and ABAC.
- How users, groups, and roles organize access.
- Privilege creep, and why access must be reviewed.

Before You Start

Read lesson 6 (authentication) and lesson 2 (AAA). Authentication proved who you are; this lesson is about what you are allowed to do once you are known. No programming is needed.

---

1. Authentication vs. Authorization

These two words are often confused, and keeping them apart is the foundation of this lesson.

- **Authentication** answers "**who are you?**" (lesson 6). It happens once, at the door.
- **Authorization** answers "**what are you allowed to do?**" It happens on every action, after you are known.

An analogy: at an airport, authentication is showing your passport to prove who you are. Authorization is your boarding pass, it says which flight and seat you may take. Proving your identity does not, by itself, let you board any plane you like.

A real system checks both. You might be perfectly, correctly authenticated as yourself, and still not be authorized to read the payroll file or delete another user's account. A huge share of real breaches come from **broken authorization**: a user who is logged in as themselves reaches data or actions they should never have been allowed, often just by changing a number in a web address. Getting authorization right is as important as getting authentication right.

2. Least Privilege

The principle of least privilege: give each person, program, and device only the access it needs to do its job, and no more.

This is the single most important idea in access control, and one of the most important in all of security. If a user only needs to read orders, they should not be able to delete customers. If a program only needs to write to one folder, it should not be able to touch the whole disk.

Why it matters so much:

- ***It limits the damage of any single failure.*** *If an account with least privilege is phished or a program is hacked, the attacker gains only that small slice of access, not the keys to everything. Least privilege is what turns a disaster into an incident.*
- ***It limits mistakes.*** *People cannot accidentally delete what they cannot touch.*
- ***It shrinks the attack surface.*** *Fewer permissions means fewer things an attacker can abuse.*

A closely related idea is the practice of ***not using an administrator account for everyday work****. Even an IT administrator should browse the web and read email as an ordinary user, and switch to their powerful account only for the specific task that needs it, so that a bad click does not happen while holding the keys to everything. Least privilege applied in time, not just in scope.*

3. Separation of Duties

Separation of duties means no single person can complete a sensitive process alone. *A critical action is split so that it needs two or more people, which stops one person acting fraudulently or by mistake.*

The classic example is money: the person who requests a payment should not also be the person who approves it. One employee raises an invoice; a different one approves the funds. To commit fraud, you would now have to corrupt two people, which is far harder, and far more likely to be noticed.

Separation of duties shows up all over security:

- *Requesting versus approving access to a system.*
- *Writing code versus approving its release to production.*
- *An administrator who can change settings, but a separate person who reviews the logs of those changes.*

It pairs naturally with least privilege: least privilege limits how much power one role has, and separation of duties ensures the most dangerous powers are split across roles. Together they are a strong defense against both insider threats (lesson 4) and honest mistakes.

4. Access Control Models

*How does a system actually decide who may do what? There are a few standard **models**. You do not need to implement them yet, but you should recognize the names and the ideas.*

Model	How access is decided	Where you see it
DAC (Discretionary Access Control)	The **owner** of a resource decides who may use it	Files on your computer; sharing a document and choosing who can view it
MAC (Mandatory Access Control)	A **central policy** decides, based on labels (for example "Secret"); users cannot override it	Military and high-security systems
RBAC (Role-Based Access Control)	Access is granted to **roles**, and people are given roles	Most businesses: "Cashier", "Manager", "Admin"
ABAC (Attribute-Based Access Control)	Access is decided by **attributes and rules** (user's department, time of day, device, location)	Large, flexible modern systems

*The two you will meet most are **DAC**, the everyday model where owners share their own files, and **RBAC**, the backbone of almost every business system. **MAC** is strict and central, used where secrecy is paramount. **ABAC** is the most flexible and powerful, deciding access from a rich set of conditions, and is common in large cloud systems. Real systems often blend them.*

5. Users, Groups, and Roles

Managing permissions for each person one by one does not scale, and quickly becomes a mess. So access is organized in layers:

- *A **user** is one account, one person (or one program or device).*
- *A **group** is a bundle of users, so you can grant access to many people at once ("Phnom Penh Store Staff").*
- *A **role** is a named set of permissions tied to a job ("Cashier" can take payments and issue refunds up to a limit).*

*The power of this is that you manage access by **job, not by person**. You decide once what a "Cashier" may do, then give the Cashier role to whoever works the till. When someone changes jobs, you change their role, and their access follows. When a new weakness or rule appears, you fix it in one place, the role, and everyone with that role is updated. This is RBAC in practice, and it is how almost every organization keeps access manageable.*

6. Privilege Creep

Access is not a one-time decision; it rots over time. ***Privilege creep*** *(also called access creep) is the slow build-up of permissions a person no longer needs.*

It happens naturally: someone joins the sales team and gets sales access; they help the finance team for a month and get finance access; they move to support and get support access, but nobody ever removes the first two. After a few years, a long-serving employee may be able to reach almost everything, which quietly destroys least privilege and makes that one account a huge prize for an attacker.

The fixes are process, not technology:

- ***Access reviews:*** *regularly check who has access to what, and remove what is no longer needed. This is called* ***recertification****.*
- ***Joiner-mover-leaver process:*** *grant access when someone joins, change it when they move roles, and promptly remove all of it when they leave. Forgotten accounts of former staff are a classic way in.*
- ***Time-limited access:*** *grant temporary access that expires automatically, so it cannot linger.*

Privilege creep is a reminder that access control is an ongoing practice, the same lesson as residual risk in lesson 3: security is never "set and forget".

7. Access Control in Practice

Put the pieces together for a small business, say an online shop:

Idea	*In practice*
Authentication	*Everyone signs in with a unique account and MFA (lesson 6)*
Authorization	*The system checks, on every action, whether this role may do it*
Least privilege	*A cashier can process orders, but cannot change prices or export the customer list*
Separation of duties	*Refunds over a limit need a manager's approval; the person who requests is not the one who approves*
Roles (RBAC)	*"Cashier", "Manager", "Admin" roles, assigned by job*
Privilege creep	*Access is reviewed every quarter, and removed the day someone leaves*

Notice that none of this requires exotic technology. It requires clear thinking about who needs what, and the discipline to keep it current. Those habits prevent a large share of real incidents: an over-powered account that gets phished, a former employee whose login still works, a

cashier who can quietly export every customer's details. Access control is where the goals of lesson 2, especially confidentiality and integrity, are enforced in daily practice. In the next lesson, we widen the view to all the kinds of control a security program uses, and how they stack into defense in depth.

Common Mistakes

1. Confusing authentication and authorization. *Being correctly logged in as yourself does not mean you are allowed to do everything. The system must check permission on every action.*

2. Giving everyone broad access "to be safe". *The opposite is true. Broad access means any single compromised account is a catastrophe. Least privilege limits the blast radius.*

3. Using an admin account for everyday work. *A bad click while holding full power is a disaster. Use an ordinary account by default; elevate only for the task that needs it.*

4. Letting one person control a whole sensitive process. *Without separation of duties, one person can commit fraud or make an unchecked mistake. Split request from approval.*

5. Granting access and never reviewing it. *Privilege creep turns long-serving accounts into skeleton keys. Review access regularly and remove what is unused.*

6. Forgetting to remove access when people leave. *A former employee's working login is a classic breach. Remove all access the day someone leaves.*

Summary

Term	Remember
Authentication vs. authorization	Who you are vs. what you may do
Least privilege	Only the access needed, nothing more; limits damage and mistakes
Separation of duties	No one person completes a sensitive process alone; stops fraud
DAC / MAC / RBAC / ABAC	Owner decides / central policy / by role / by attributes and rules
Users, groups, roles	Manage access by job, not by person
Privilege creep	Permissions pile up over time; review and remove them

Check Your Understanding

1. In one sentence each, what do authentication and authorization answer?

2. State the principle of least privilege, and give one reason it matters.
3. What is separation of duties, and what kind of harm does it prevent?
4. Match each model to its idea: DAC, MAC, RBAC, ABAC.
5. Why manage access by role instead of per person?
6. What is privilege creep, and name one process that fixes it.
7. Why is using an administrator account for everyday web browsing a bad idea?

Answers

1. Authentication answers "who are you?"; authorization answers "what are you allowed to do?".
2. Give each person, program, and device only the access it needs and no more. It matters because it limits the damage of any compromised account or mistake (and shrinks the attack surface).
3. No single person can complete a sensitive process alone; a critical action needs two or more people. It prevents fraud and unchecked mistakes.
4. DAC: the resource's owner decides. MAC: a central policy decides from labels; users can't override. RBAC: access is granted to roles, and people are given roles. ABAC: access is decided by attributes and rules.
5. Because you decide once what a job may do, then assign that role to people; changes are made in one place and follow people as they change jobs, which is manageable and consistent.
6. The slow build-up of permissions a person no longer needs. Fixes: regular access reviews (recertification), a joiner-mover-leaver process, or time-limited access.
7. Because if a bad click or malware strikes while you hold full administrative power, it can affect everything. Browsing as an ordinary user limits the damage of that moment.

Practice

Think about an organization you know, or imagine a small online shop with these jobs: cashier, shift manager, owner, and a part-time bookkeeper.

1. Write a simple **role** for each job: two or three things that role is allowed to do.
2. Identify one **sensitive action** (such as issuing a large refund or exporting all customer data) and apply **separation of duties**: who requests it, and who approves it?
3. Name one place **privilege creep** could happen, and the process you would use to catch it.

A Worked Example

1. *Cashier: take payments, issue small refunds, look up an order. Shift manager: all of the above, approve larger refunds, open and close the till. Owner: everything, plus manage accounts. Bookkeeper: view sales reports and export accounting data, but not change prices or process orders.*
2. *Sensitive action: a refund over $50. The cashier requests it; the shift manager approves it. The person who requests is never the one who approves.*
3. *Privilege creep: the bookkeeper once covered for a manager and kept manager access. Catch it with a quarterly access review that removes any permission not matching the person's current job.*

Course mark icon from Material Symbols by Google, Apache License 2.0. NIST, CIS, ISO, OWASP, and MITRE ATT&CK are the trademarks of their respective owners, used here only to refer to their published work.